

Project II: Convolutional Neural Networks & Computer Graphics

Andrew Bonilla

1 Implementation Overview

1.1 Data Processing

We were given a training dataset of 750 colored JPEG images, of 128×128 pixels. Images were loaded via OpenCV, converted appropriately from BGR to RGB, normalized, and stored as tensors of shape $750 \times 3 \times 128 \times 128$.

The dataset was augmented by a factor of 10, resulting in 7500 training images after simultaneous combinations of operations (horizontal flipping, random cropping, scaling by a random scalar).

Images were finally converted from RGB to $L^*a^*b^*$ color space using OpenCV. This separated luminance (L^*) from chrominance (a^* , b^*), which was useful for colorization problems. Both the augmented dataset and the $L^*a^*b^*$ dataset were saved in separate folders (augmented, L^* , a^* , b^*).

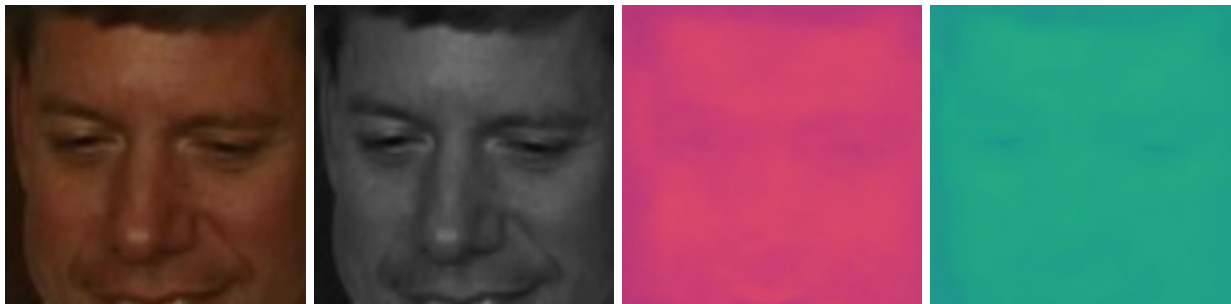


Figure 1: Example image showing (a) original RGB, (b) L^* channel, (c) a^* channel, (d) b^* channel.

1.2 Regressor

I implemented a CNN regressor to predict the mean chrominance values of a given grayscale image. The network has 7 modules, where each has a Conv2d layer followed by a ReLU activation. Each convolution uses stride 2 and padding 1 to halve the spatial dimensions at each step, shrinking the input from 128×128 to 1×1 ($128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$). All hidden layers use 3 feature maps. A final fully connected layer maps the output to 2 scalars representing mean a^* and mean b^* . The L^* channel input was scaled from $[0, 100]$ to $[0, 1]$ before being passed to the network.

The regressor was trained using MSE loss and the Adam optimizer with a learning rate of 0.001 for 100 epochs. It followed with a decrease in training loss.

1.3 Colorization Network

The colorization network is an encoder/decoder structure with $N=5$ downsampling and upsampling layers. The encoder reduces spatial resolution and increases feature map depth. The decoder builds spatial resolution via nearest-neighbor upsampling followed by convolution layers.

The input for the network is an L^* channel ($1 \times 128 \times 128$). The output is two chrominance channels ($2 \times 128 \times 128$) for a^* and b^* . Batch normalization is applied after each layer. To fulfill the project requirement, training was performed using mini-batches of 10 images. For the extra credit, a Tanh activation

function was eventually used in the final layer, with a^* and b^* targets rescaled to $[-1, 1]$ during training and returned to $[0, 1]$ during testing.

The dataset was split into 90% training (original 675 images, augmented 6,750) and 10% testing (75 images, not augmented). The model was trained using MSE loss and the Adam optimizer with a learning rate of 0.001 for 100 epochs.

1.4 GPU Computing

The colorization network was sped up using CUDA GPU computing. The model and all data tensors were moved to the GPU using PyTorch’s `.to(device)` method. To measure the acceleration, inference was performed on 100 test images on both CPU and GPU. The GPU achieved a speedup of 6.52x over the CPU.

2 Colorization Results



Figure 2: Example colorization results on the faces test set.

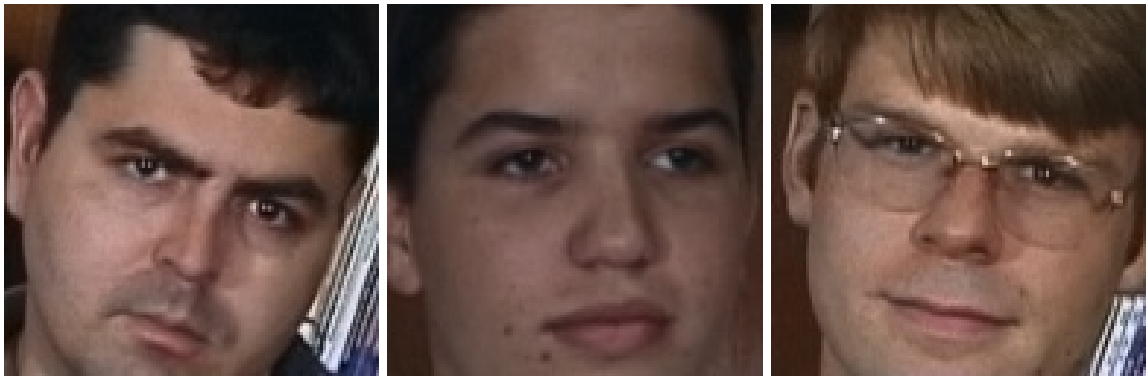


Figure 3: Example colorization results on the faces training set.

All colorization results shown were produced using the GPU implementation on an NVIDIA T4 GPU (these are the GPU, testing, and training results).

3 Hyperparameter Tuning

Three variants of the colorization network were trained and evaluated, differing in the number of feature maps in the encoder and decoder:

Architecture	Encoder Channels	Test MSE
Small	$1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 256$	0.000352
Medium	$1 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$	0.000192
Large	$1 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512 \rightarrow 512$	0.000221

Table 1: Hyperparameter tuning results on the faces dataset.

The medium architecture achieved the best test MSE of 0.000192. The large and small architecture performed worse than the medium option, likely due to overfitting given the relatively small dataset size. Despite having the lowest training loss (0.000154), the small architecture had the highest test MSE, suggesting overfitting. The medium architecture achieved the best balance between training performance and generalization. So, the medium architecture was selected for the transfer learning task.

4 Transfer Learning

4.1 Strategy

The pretrained medium architecture was fine-tuned on the NCDataset + ColorfulOriginal datasets of 721 paired grayscale and color images of fruits and vegetables over 20 categories. The encoder weights were frozen to preserve low-level features learned from the faces dataset, and only the decoder was fine-tuned for 50 epochs using Adam optimization with a learning rate of 0.0001.

4.2 Evaluation Results

Dataset	Test MSE
Faces	0.000211
NCD (Fruits & Vegetables)	0.010744

Table 2: Colorization performance across datasets.

The higher MSE on NCD could reflect a greater difficulty of colorizing diverse natural objects compared to faces. Visual inspection showed reasonable colorization with some fruit-appropriate colors, though a slight bias toward the face training domain was visible in some images.



Figure 4: Example colorization results on the NCD dataset after transfer learning.

5 Extra Credit

5.1 GPU Speedup

As mentioned before, inference was timed on 100 face images using both CPU and GPU implementations. The GPU completed inference in 0.0610 seconds compared to 0.3975 seconds on CPU, yielding a speedup of 6.52x.

5.2 Tanh Activation

Following the approach of Iizuka et al. (2016), a Tanh activation was added as the final layer of the decoder. The a^* and b^* training targets were rescaled from $[0, 1]$ to $[-1, 1]$ to match the Tanh output range, and predictions were rescaled back to $[0, 1]$ during testing.

Model	Test MSE
Without Tanh	0.000211
With Tanh	0.000313

Table 3: Effect of Tanh activation on test MSE.

6 How to Run

All scripts should be run from the terminal with the virtual environment activated:

```
source /DATA/andrewbonilla/envs/dlcn/bin/activate
```

You must also run this in case it is not installed in AWS:

```
pip install opencv-python
```

The following directory structure is expected on the server for `/DATA/andrewbonilla/`:

<code>face_images/</code>	(face dataset)
<code>NCDataset/</code>	(NCD grayscale images)
<code>NCColorful/</code>	(NCD color images)

Running `regressor.py` or `train.py` will automatically regenerate the augmented and LAB datasets from the original `face_images/` directory and save them to the `augmented/`, `L/`, `a/`, and `b/` folders. These folders are also included in the submission for reference, but they only include a few images rather than the whole set. All other folders and files are generated automatically by running the scripts.

- **regressor.py** : Loads the dataset, trains and runs the mean chrominance regressor:

```
python3 /DATA/andrewbonilla/regressor.py
```

- **train.py** : Trains the full colorization network on the faces dataset. Requires `face_images/` in `/DATA/andrewbonilla/`. Saves model to `colorizer.pth`:

```
python3 /DATA/andrewbonilla/train.py
```

- **test.py** : Evaluates the trained model on the test set, prints MSE, and saves colorized images to `colorized_results/` and `colorized_train_results/`:

```
python3 /DATA/andrewbonilla/test.py
```

- **transfer.py** : Fine-tunes the pretrained model on the NCD dataset and saves colorized results to `ncd_results/`. Requires `NCDataset/` and `NCColorful/` in `/DATA/andrewbonilla/`:

```
python3 /DATA/andrewbonilla/transfer.py
```

- **gpu_speedup.py** : Measures and reports GPU vs CPU inference speedup:

```
python3 /DATA/andrewbonilla/gpu_speedup.py
```

7 Notes

The dataset split uses a fixed random seed (`torch.manual_seed(42)`) to ensure there is reproducibility across `train.py` and `test.py`.